Improving train connectivity over multiple cellular networks by predicting handover events

Darragh O'Keeffe
Final Year Project – BSc in Computer Science
Supervisor: Prof. Holger Claussen
Co-Supervisor: Dr. Kariem Fahmi
Department of Computer Science
University College Cork

April 19, 2026

Abstract

Reliable mobile connectivity on moving trains remains a challenge due to frequent transitions between cells, which can degrade throughput and increase latency for users onboard. Stable onboard internet connectivity is critical for maintaining *Quality of Service (QoS)* in high-mobility environments, where disruptions can significantly impact latency-sensitive applications such as video calls, video live streaming and gaming.

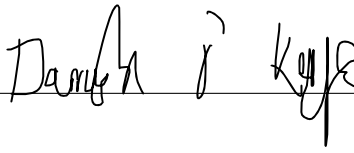
The objective is to create a machine learning model that can predict imminent handover events, using features derived from signal, spatial and temporal data, with the aim to improve on-board internet connectivity for passengers. A dataset was created using real world measurements collected on the *Dublin Area Rapid Transport (DART)*. Cellular network signal metrics were combined with spatial and temporal features derived from *GPS* data. Data was collected across *9* trips on the rail line from *Tara Street Station* to *Howth Station* and back, which generated approximately *35,000* examples. A *Mixture of Experts (MoE)* model was chosen for its suitability to the heterogeneous nature of rail routes, capturing varying network behaviour in different locations and conditions. This allowed the system to specialise in distinct connectivity patterns observed along this route. The final model achieved a *99 percent event detection rate, 45 percent precision, 94 percent recall, and 70 percent PR-AUC* for event prediction. While signal features contribute most significantly to model performance, results also show that handover events consistently cluster at stable geographic locations throughout repeated journeys, with spatial features contributing also to model performance. These findings demonstrate that handover behaviour is highly predictable in structured mobility scenarios, supporting the use of route aware predictive models for anticipatory network optimisation. This work provides a foundation for more reliable and intelligent connectivity in high-mobility environments.

Declaration of Originality

In signing this declaration, you are conforming, in writing, that the submitted work is entirely your own original work, except where clearly attributed otherwise, and that it has not been submitted partly or wholly for any other educational award.

I hereby declare that:

- this is all my own work, unless clearly indicated otherwise, with full and proper accreditation;
- with respect to my own work: none of it has been submitted at any educational institution contributing in any way to an educational award;
- with respect to another's work: all text, diagrams, code, or ideas, whether verbatim, paraphrased or otherwise modified or adapted, have been duly attributed to the source in a scholarly manner, whether from books, papers, lecture notes or any other student's work, whether published or unpublished, electronically or in print.

Signed: 

Date: 19/04/26

Acknowledgements

I would like to express my sincere gratitude to Prof. Holger Claussen and Dr. Kariem Fahmi for sharing their expertise and offering their guidance over the last number of months. Their insights and willingness to generously dedicate their time were paramount to the completion of this work. Finally, I would like to thank my family for their continued support throughout my studies.

Contents

Abstract

Declaration

Acknowledgements

1 Introduction **1**
- 1.1 Problem Context . . . 1
- 1.2 Motivation . . . 2
- 1.3 Report Outline . . . 2

2 Analysis **4**
- 2.1 Related Work / Literature Review . . . 4
- 2.2 Objective and its context . . . 6
- 2.3 Deliverables and Requirements . . . 7
- 2.4 Constraints . . . 8

3 Design **9**
- 3.1 Methodology . . . 9
- 3.2 Data Collection . . . 10
- 3.3 Data Processing Design . . . 10
  - 3.3.1 Timestamp Alignment . . . 10
  - 3.3.2 Trip Segmentation . . . 11
  - 3.3.3 Cumulative Distance Representation . . . 12
  - 3.3.4 Handover Detection . . . 12
- 3.4 Feature Engineering and Selection Design . . . 12
  - 3.4.1 Signal Features . . . 13
  - 3.4.2 Spatial Features . . . 15
- 3.5 Model Development Design . . . 16
- 3.6 Model Evaluation Design . . . 20

4 Implementation **22**
- 4.1 System Overview . . . 22
- 4.2 Data Collection Implementation . . . 22
- 4.3 Data Ingestion and Preprocessing . . . 23
  - 4.3.1 Trip Segmentation . . . 24
- 4.4 Data cleaning . . . 24
  - 4.4.1 Ingestion-Stage Cleaning . . . 25
  - 4.4.2 Post-Segmentation Cleaning . . . 25
- 4.5 Data exploration . . . 26

4.6 Feature Engineering . . . 27
4.7 Model Training and Development Implementation . . . 28
4.7.1 Problem Formulation . . . 28
4.7.2 Model Architecture . . . 28
4.7.3 Training Procedure . . . 29
4.7.4 Cross-Validation Strategy . . . 31
4.7.5 Threshold Selection . . . 31
4.8 Model evaluation Implementation . . . 31

5 Evaluation **33**
5.1 Experimental Setup . . . 33
5.2 Problem Formulation Comparison . . . 33
5.2.1 Time-to-Handover Regression . . . 33
5.2.2 Binary Classification (Next 1 Second) . . . 35
5.2.3 Binary Classification (Next Sample) . . . 35
5.2.4 Tolerance-Based Classification . . . 35
5.3 Tolerance Window Analysis . . . 35
5.4 Model Performance . . . 36
5.5 Threshold Sensitivity . . . 37
5.6 Feature Contribution Analysis . . . 38
5.7 Expert Utilisation in MoE . . . 39
5.8 Key Findings . . . 39
5.9 Limitations . . . 40

6 Conclusion **40**
6.1 Discussion . . . 40
6.2 Answer to Research Question . . . 41
6.3 Contribution . . . 42
6.4 Future Work . . . 42
6.5 Reflection . . . 43
6.6 Project Timeline . . . 43

7 Appendix **44**

List of Figures

1 Schematic of a train moving across overlapping cellular coverage areas (Cell A, Cell B, Cell C). As the train progresses, signal strength degrades and a handover is triggered at the cell boundary 1
2 Illustration of the *A3* event trigger mechanism, showing RSRP/RSRQ measurements from the serving and neighbour cells over time. A handover is triggered once the neighbour cell signal exceeds the serving cell by the configured offset for the duration of the Time-to-Trigger (TTT) period [7]. 4
3 End-to-end system architecture illustrating the full pipeline from raw data collection through preprocessing, feature engineering, model training and evaluation. 9
4 GPS overlay of the recorded rail route from Tara Street to Howth, visualised in *Google Earth Pro*. 11
5 Overlay of cumulative distance across multiple trips, showing the degree of structural alignment between processed journeys. Variation in trip duration and slope reflects differences in train speed and dwell time at stations across sessions, rather than inconsistencies in segmentation. 13
6 Pearson correlation heatmap of signal features 14
7 Handover density measured at each bin along the track showing clustering behaviour. Captured on *Google Earth Pro* 15
8 High base station density in Tara Street area, illustrating the dense urban deployment environment in which more frequent handovers are expected. Adapted from [9] 18
9 Low base station density in Howth, illustrating the sparse urban deployment environment in which less frequent handovers are expected. Adapted from [9] 19
10 Mixture of Experts architecture for handover prediction. GPS and signal inputs are routed by a gating network across n parallel expert networks, each potentially specialising on distinct regimes along the route. Expert outputs are combined via learned weights to produce a final prediction. Adapted from [8] 20
11 Full implementation pipeline showing raw inputs to final predictions 23
12 Tolerance-based label construction for $k = 5$. Only samples preceding the handover event are labelled as positive. 29
13 Training loss per epoch for the MoE handover classifier on a representative fold (Fold 1). The curve demonstrates rapid initial convergence followed by diminishing improvement beyond approximately epoch 35, supporting the choice of 50 epochs as a sufficient training number. The dashed line indicates the early stopping boundary. 30

14 Precision-recall curve for the MoE handover classifier, aggregated across all 19 cross-validation folds. The red marker indicates the chosen operating point at threshold $t = 0.01$, yielding precision 0.44 and recall 0.95. The dashed baseline represents a random classifier at the dataset's positive class prevalence. 32
15 SMAPE across time-to-handover ranges 34
16 Permutation feature importance averaged across all folds, expressing the contribution of each feature to model performance (measured by decrease in PR-AUC). The model is primarily driven by signal quality and mobility features, with minimal reliance on identifier based inputs. The error bars will also demonstrate that feature importance experiences variability across trips. 37

List of Tables

1 Fundamental characteristics of problem formulations 7
2 Strengths and weaknesses of problem formulations 7
3 Final feature set used for model training, including brief descriptions and categorisation by feature domain. 16
4 Evaluation metrics used to assess classification model performance. 21
5 Comparative performance across formulations 36
6 Comparative performance across different tolerance windows 36
7 Per-fold performance variation across 19 cross-validation folds. 36

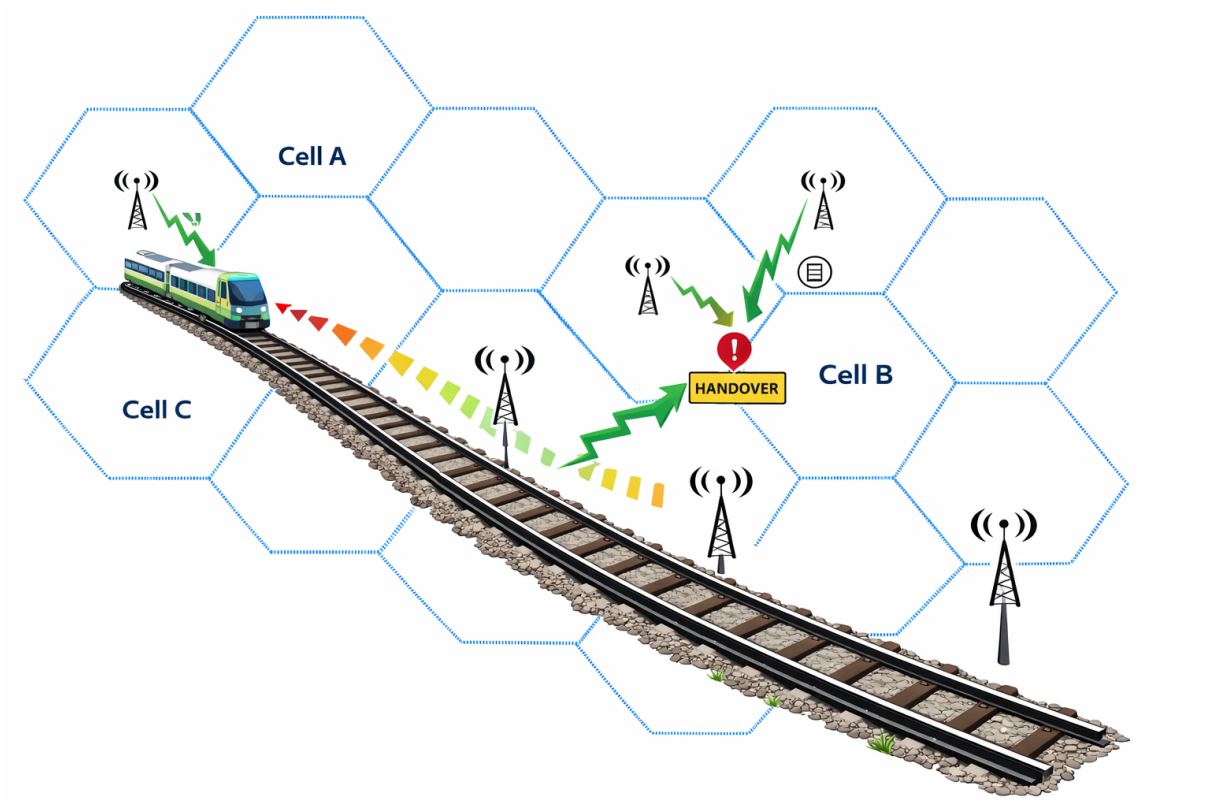


Figure 1: Schematic of a train moving across overlapping cellular coverage areas (Cell A, Cell B, Cell C). As the train progresses, signal strength degrades and a handover is triggered at the cell boundary

1 Introduction

1.1 Problem Context

Reliable cellular connectivity is essential for contemporary railway transport routes, where passengers (users) should expect a minimally interrupted connection over mobile data. From a practical perspective, some examples could include taking phone calls, live streaming video and gaming. In a highly mobile environment such as a rail track, a train moves through in and out of coverage areas, the *User Equipment (UE)* must transition between base stations in a process known as a *handover*. While Figure 1 presents handovers occurring at cell boundaries for illustrative purposes, real world handover decisions are not strictly tied to geographic boundaries. Instead, they are influenced by factors such as signal quality (e.g., RSRP, SINR), network load, interference, and operator-defined thresholds, meaning that handovers may occur before or after a boundary is reached. These handovers are critical to maintaining a sufficient *Quality of Service (QoS)*; unstable han-

dovers can result in packet loss, latency spikes, and/or service interruption [1]. For normal non-realtime web traffic, this might be acceptable , but high latency-sensitive applications running on user equipment (UE) are particularly affected by unstable handovers. For example, a passenger that is live-streaming a video may experience significant frame drops, while online gaming applications can suffer from increased latency and degradation of responsiveness.

1.2 Motivation

On-board internet connectivity typically relies on cellular links as they have sufficient coverage along rail tracks. They often use solutions that combine multiple cellular connections to increase capacity and reliability *(e.g., Irish Rail uses [4]Peplink; a multi-WAN bonding solution , which aggregates and manages multiple links simultaneously)*. These solutions will load-balance passenger traffic across the available links to increase overall capacity. However, these approaches are largely reactive and do not account for imminent handover events that may degrade link quality. Handover decisions are typically based on key signal quality indicators such as *Reference Signal Received Power (RSRP)*, *Reference Signal Received Quality (RSRQ)*, *Signal-to-Noise Ratio (SNR)* and *Signal-to-Noise-plus-Interference Ratio (SNIR)*. While it is not possible to control or influence the occurrence of these handover events, user traffic can be prepared for potential disruption caused during a handover. A proactive strategy involves redirecting latency-sensitive traffic away from links predicted to experience a handover, instead utilising more stable links. In addition, traffic may be briefly replicated across multiple links to mitigate degradation of QoS during the handover event. These approaches enable more seamless real-time communication. Machine learning approaches for handover optimisation have gained increasing attention in recent years [2]. In the case of a train travelling on a railway line from one point to another, movement is highly repeatable. Trains follow the same physical track each time, traversing the same geographical areas with often repetitive network conditions. This suggests that, in addition to instantaneous signal measurements, spatial context and historical movement patterns may contain useful predictive information. This project aims to predict imminent handover events using a machine learning model based on both real-time signal measurements and learned route-specific patterns, enabling proactive network adaptation rather than reactive response.

1.3 Report Outline

This report is structured as follows. The second section following from the introduction, *Analysis*, reviews related work on cellular handover mechanisms and existing machine learning approaches to mobility prediction, highlighting the limitations of signal-threshold-based methods.

The third section, *Design,* presents the system design, including the overall methodology, data sources, processing pipeline, feature engineering strategy, and the formulation of the prediction task. This section outlines the key design decisions and rationale behind the proposed approach.

The *Implementation* section describes the implementation of the system, detailing how network and GPS data are collected, aligned, and processed, as well as how features are constructed and models are developed, trained and evaluated.

The fifth section, *Evaluation*, assesses the proposed approach through a series of experiments, presenting key results and analysing model performance using appropriate metrics. The effectiveness of spatial and signal-based features is examined, along with limitations of the approach.

Finally, the *Conclusion* finishes the report by summarising the main findings, discussing their implications, and outlining directions for future work.

2 Analysis

2.1 Related Work / Literature Review

A cellular network divides a geographic area into a set of contiguous regions known as *cells*, each served by a dedicated *Base Station (BS)*, referred to in *Long Term Evolution (LTE)* as an *eNodeB*. Each eNodeB manages radio resources within its cell and handles communication between the network core and any mobile devices, referred to as *User Equipment (UE)*, currently attached to it as seen in [6]

In LTE networks, a mobile device maintains connectivity by communicating with a *Base Station (BS)*. As a device moves through space, it periodically measures the signal quality of nearby base stations using metrics such as *Reference Signal Received Power (RSRP)*, *Reference Signal Received Quality (RSRQ)*, and Signal-to-Noise Ratio (SNR). When signal conditions from a neighbouring cell become sufficiently more favourable than the serving cell, a handover is triggered, transferring the connection to the new base station [5]. The handover process is illustrated in Figure 1 and Figure 2. Handover management

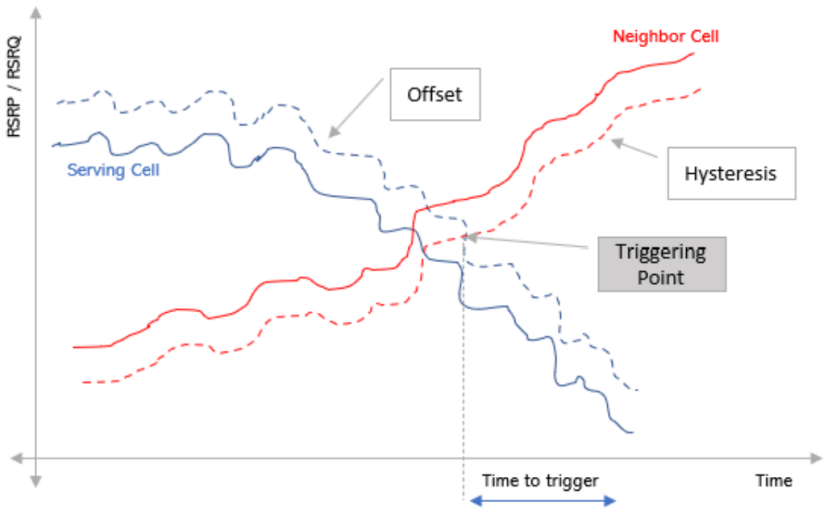


Figure 2: Illustration of the *A3* event trigger mechanism, showing RSRP/RSRQ measurements from the serving and neighbour cells over time. A handover is triggered once the neighbour cell signal exceeds the serving cell by the configured offset for the duration of the Time-to-Trigger (TTT) period [7].

is a major challenge in mobile wireless networks, particularly in high-mobility environments such as railway systems, where connectivity conditions change rapidly as paths are traversed. Furthermore, while railway routes are fixed and repeatedly traversed, the wireless circumstances along a given route are not static. Interference and network load

will invariably change across journeys. This introduces variability that makes purely reactive approaches unreliable. As mentioned in the introduction, standard handover decision mechanisms are primarily reactive, relying on signal strength thresholds, hysteresis margins, and event-based triggers such as *A3* events in LTE networks. These approaches are quite rudimentary but also ubiquitous, but they depend heavily on instantaneous signal measurements and do not incorporate any context of future link conditions or mobility patterns. As a result, they are limited in their ability to proactively mitigate disruptions experienced during handover events.

Abundant work has focused on mitigating the impact of handovers without attempting to predict them. In particular, multipath and redundancy-based approaches have been proposed to improve reliability in train-to-ground communication systems. For example, extensions of *Multipath TCP (MPTCP)* replicate packets across multiple available links to ensure continuity of service, particularly for mission-critical railway applications. While such approaches can significantly reduce packet loss, they introduce considerable redundancy, leading to inefficient use of network capacity and increased congestion under high load conditions, as mentioned in [2]. This trade-off between reliability and efficiency is one motivation for the exploration of predictive approaches that can apply more intelligent and selective signal degradation mitigation strategies.

More recent work has subsequently shifted towards predicting handover events using machine learning techniques. *Fahmi* and *Leith* propose a neural network model that predicts whether a handover will occur within a short time window using features derived solely from client-side logs [1]. Their results demonstrate that combining temporal and contextual features significantly improves prediction performance compared to predicting solely from signal data. Crucially, they show that accurate prediction facilitates adaptive packet replication, achieving similar loss reduction to full redundancy while using substantially fewer resources.

However, their work also highlights a key challenge: handover behaviour can vary significantly even across repeated traversals of the same route [1]. This variability arises due to factors such as varying speed and acceleration, environmental interference, and complex interactions within the wireless medium. As a result, purely signal-based or short-term temporal models may fail to generalise across sessions on the same route.

To address this, several studies have explored location-aware and mobility-based prediction strategies. These approaches incorporate additional contextual information such as GPS location, velocity, and historical mobility patterns to improve predictive accuracy. For example, prior work has shown that connectivity maps can be constructed using historical data, allowing systems to anticipate future outages based on predicted location and pre-fetch data accordingly [3]. Such approaches are particularly well-suited to structured environments like railway systems, where movement is constrained and routes are repeatedly traversed.

Across the literature, several consistent observations come to light. First, signal met-

rics alone are insufficient for reliable prediction, as they fail to capture the underlying spatial and temporal context. Secondly, while spatial patterns in handover behaviour do exist, they are certainly noisy and influenced by abstract factors, making them challenging to model directly. Finally, there exists a pervasive trade-off between prediction accuracy and prediction-based outcomes: false positives lead to unnecessary resource usage, generally bloating links, while false negatives preserve a baseline of poor QoS.

Despite these refinements, there is a gap for explicitly leveraging route-specific behaviour in railway environments. Specifically, the idea of aligning and aggregating historical data along a persistent spatial representation of a track, and learning location dependent handover behaviour remains under explored. This project builds on the existing body of work by adopting a route-specific and location-aware modelling approach, combining temporal, spatial, and signal features, to better capture the underlying patterns of handover dynamics in a fixed-route setting.

2.2 Objective and its context

The objective of this project is to build a model that can reliably predict when a handover is going to happen. These predictions are continuously updated as new network measurements are recorded and sampled, when network and spatial conditions are re-evaluated. This model is trained on sampled LTE cellular modem data which includes both signal and spatial features. There are several ways in which the problem of predicting handovers can be formulated. Each formulation captures a different perspective on the prediction task, ranging from univariate regression to discrete classification.

1. **Time-to-Handover Regression**
   Predicts the time remaining (in seconds) until the next handover event. This provides fine-grained insight, but is sensitive to noise, particularly near the handover point. As a rule of thumb, prediction difficulty increases proportionate to the duration of time to handover estimates (often though there is noise around quick successive handovers. This complexity increases due to higher variability.)

2. **Binary Classification (Next X Second)**
   Predicts whether a handover will occur within the next x number of seconds. This reframes the problem into a context that aligns well with real-time system requirements.

3. **Binary Classification (Next Sample/Moment)**
   This formulation is more demanding than the binary `next_1_second` approach. Asking whether a handover occurs at the next sample, introduces additional uncertainty due to the irregular sampling intervals, making the prediction less consistent and more sensitive to timing variability.

4. **Binary Classification with Tolerance Window (k seconds)**
   Extends the *next_sample* formulation by allowing a tolerance window (e.g., 2 to 5 seconds), where predictions are considered correct if they fall within a defined range before the actual handover, but strictly not after. A prediction that is later than the actual handover should not be rewarded. This method improves robustness at the cost of partial practical utility

Formulation	**Output Type**	**Practical Relevance**
Time-to-Handover Regression	Continuous (seconds)	Limited
Binary (Next 1s)	Discrete (0/1)	High
Binary(Next Sample) (k)	Discrete (0/1)	Moderate
Binary with Tolerance (k)	Discrete (0/1 with window)	High

Table 1: Fundamental characteristics of problem formulations

Formulation	**Strengths**	**Weaknesses**
Time-to-Handover Regression	Fine-grained prediction	High error near handover
Binary (Next 1s)	Simple, real-time aligned	Sensitive to timing
Binary (Next Sample)	Simple and intuitive formulation	Only perfect predictions are rewarded
Binary with Tolerance (k)	More robust, improved recall	Reduced temporal precision

Table 2: Strengths and weaknesses of problem formulations

Finally, a more implicit objective of this project is to contribute to the body of research in the area of mobility management in cellular networks. There are clear benefits in fulfilling these objectives. Enabling proactive network behaviour results in mitigation of the data loss experienced during handovers, and ensures seamless, real-time communication. More broadly, the approach provides a foundation for more intelligent mobility management in future wireless systems.

Critical performance of each formulation and model is evaluated using appropriate metrics, with a particular focus on classification-based approaches. More familiar metrics such as *recall, precision, and F1-score* are used to assess predictive performance, but also more problem specific metrics such as *event detection* and *mean expert usage distribution*. There must be an emphasis placed on recall due to the importance of correctly detecting handover events, which is discussed further in *implementation*.

2.3 Deliverables and Requirements

The primary deliverable of this project is a machine learning model pipeline that produces reliable handover predictions from LTE cellular modem data. The pipeline must process

an input of tabular data including signal and spatial features sampled from the chosen rail line and produce a prediction that is either a univariate time-to-handover estimate, or a binary classification label. This depends on the formulation of the problem under evaluation. The pipeline is trained and tested on a manually collected, labelled dataset.

A successful pipeline must demonstrate in particular strong predictive performance on the chosen rail route, across the classification-based formulations. Particular emphasis is placed on recall, given the critical cost of undetected handover events.

The secondary deliverable is the final, merged dataset that captures all separate data collection sessions into a single, fully clean, and consistently formatted training and testing resource. The dataset itself is a non-trivial deliverable in its own right, as it involves multiple data collection sessions in Dublin. The dataset must also be sufficiently large and diverse to facilitate useful model training, validation, and testing, and must contain the correct raw signal features (such as RSRP, RSRQ, and SINR) and spatial features (such as coordinates).

2.4 Constraints

Several constraints present themselves in this work. The volume of data is restricted by the time available for collection, and the practicalities of manual logging, which may limit the diversity of handover scenarios represented in the training set.

With respect to time, the project must be completed within the timeframe of a single academic year. This constrains the extent to which unique model architectures can be explored, evaluated and tuned, and limits the number of problem formulations that can be evaluated in depth. Priority is allocated to the binary classification formulations, which offer the greatest practical relevance as seen in [1].

3 Design

3.1 Methodology

To reiterate the ambition of this work, the primary objective is to design a predictive model capable of identifying imminent cellular handover events trained on real-world measurements, collected in a fixed high mobility environment cellular environment. The model should make constant predictions at the rate that signal and spatial data are both sampled. The overall system design follows an end-to-end pipeline, as shown in Figure 3.



Figure 3: End-to-end system architecture illustrating the full pipeline from raw data collection through preprocessing, feature engineering, model training and evaluation.

An important methodological change occurred during the early stages of this project. Initial approaches aimed at developing a more generalized model, capable of predicting handover behaviour across arbitrary environments. Alternatively, this would mean building a model that can make predictions on any high-mobility rail environment. However, exploratory analysis demonstrated that wireless network conditions are highly dependent on location specific factors such as cell infrastructure layout. This variability limits the effectiveness of generalised models, particularly when trained on relatively small datasets.

Bearing this insight in mind, the problem was reframed as a route-specific learning task. Railway networks are mobility environments, such that trains repeatedly traverse fixed physical routes repetitively. Although instantaneous signal conditions fluctuate between journeys due to load capacity for example, the underlying spatial patterns of connectivity remain largely persistent across multiple journeys of the same route. In light of this understanding, historical observations can be leveraged in predicting future behaviour.

It is true to suggest that this design decision limits the system in some functional value, when predicting for only a singular, fixed high-mobility environment. However, in practicality, this system is transferable onto any fixed railway route. Performance can improve progressively as learnable data is collected and becomes available for that specific track. The system is therefore trained on two primary feature domains:

- **Signal features**, capturing instantaneous network conditions
- **Spatial features**, capturing positional context along the route

This structured representation serves as the basis for the feature engineering strategy and is constantly reflected throughout the system design.

Although the reader could assume that temporal features might be assumed essential, they are redundant in this context. A temporal domain is not included, as spatial features comprehensively reflect time related changes along the route. A typical journey might fluctuate in relation to time duration and speed, but does not fluctuate in distance covered.

3.2 Data Collection

Data collection was conducted on the Dublin Area Rapid Transport (DART) line between Tara Street and Howth station. This route was selected due to it's moderate length, and exposure to varied network conditions, which includes both dense urban and coastal segments.

LTE modem logs were recorded on average at approximately one second intervals, although there were some fluctuations. Each sample includes signal metrics such as RSRP, RSRQ, and SNR, along with network identifiers including *cell ID* and *tracking area code (TAC)*.

GPS data was collected independently, with each GPS entry containing timestamped latitude and longitude coordinates. These spatial measurements are paramount in aligning network observations with physical positional context along the route.

To provide spatial context for the dataset, the full GPS trace of the recorded route is visualised in Figure 4. A total of nine trips were recorded. In most sessions, three LTE modems of distinct network operators were used. The operators under observation are *Vodafone, Three* and *Meteor*. Each modem is treated as an independent observational avenue, which proved to be an effective way to increase the dataset size without requiring additional physical journeys. This configuration also enables analysis of operator specific behaviour.

3.3 Data Processing Design

Raw modem and GPS data are noisy and subsequently require extensive preprocessing before they can be loaded into a data-frame, and used for model training. The data processing pipeline performs timestamp alignment, trip segmentation, spatial transformation, filtering out noise and event labelling.

3.3.1 Timestamp Alignment

GPS and modem data samples are aligned using nearest-neighbour timestamp matching with a tolerance of one second. This ensures that each modem sample is associated with a spatial coordinate. On rare occasions, some signal samples occur during short GPS logging



Figure 4: GPS overlay of the recorded rail route from Tara Street to Howth, visualised in *Google Earth Pro*.

gaps and are excluded as they are deemed useless for training without their associated spatial context.

3.3.2 Trip Segmentation

Trip segmentation proved to be one of the more technically challenging aspects of the system. Early approaches relied on speed thresholds and destination dwell time detection, but these methods were highly sensitive to GPS noise and inconsistent sampling, resulting in inaccurate segmentation.

The final segmentation strategy is based on spatial proximity to known terminal stations. A trip is defined as beginning when the train enters a defined radius around the start station, and then ending upon arrival at the same proximity to the opposite terminal. This approach is significantly more robust, as it relies on more stable geographic information rather than noisier features.

Some important additional preprocessing steps include filtering of GPS drift and non trips (e.g getting off the DART and leaving the station in between trips)

3.3.3 Cumulative Distance Representation

To enable spatial context across multiple journeys, a cumulative distance feature is introduced. This is computed by summing incremental distances between consecutive GPS coordinates for each trip using the *Haversine* formula (see below).

$$d = 2R \cdot \arcsin\left(\sqrt{\sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\Delta\lambda}{2}\right)}\right) \tag{1}$$

where d is the great-circle distance, R is the Earth's radius (6371 km), ϕ_1, ϕ_2 are the latitudes in radians, λ_1, λ_2 are the longitudes in radians, and $\Delta\phi$, $\Delta\lambda$ are their respective differences.

Latitude and longitude values are converted from degrees to radians prior to applying the *Haversine* formula. Direct *Euclidean* displacement is not suitable for GPS-based coordinates, as latitude and longitude represent positions on a spherical surface. The Haversine formula instead provides an accurate measure of distance travelled along the Earth's surface.

This feature enables that all individual trips are aligned along a common spatial context, enabling the model to learn historical location-dependent patterns that are consistent across journeys. To verify consistency across journeys after preprocessing, cumulative distance profiles for multiple trips are overlaid in Figure 5.

3.3.4 Handover Detection

Handover events are detected based on changes in serving cell:

$$\text{handover} = (\text{cell_id} \neq \text{previous_cell_id})$$

This definition is applied independently across all three modems to provide a consistent labelling technique across the entire dataset.

3.4 Feature Engineering and Selection Design

Feature engineering is an effective way of improving predictive performance. These new features are reliant on the quality and relevance of correctly chosen input features.

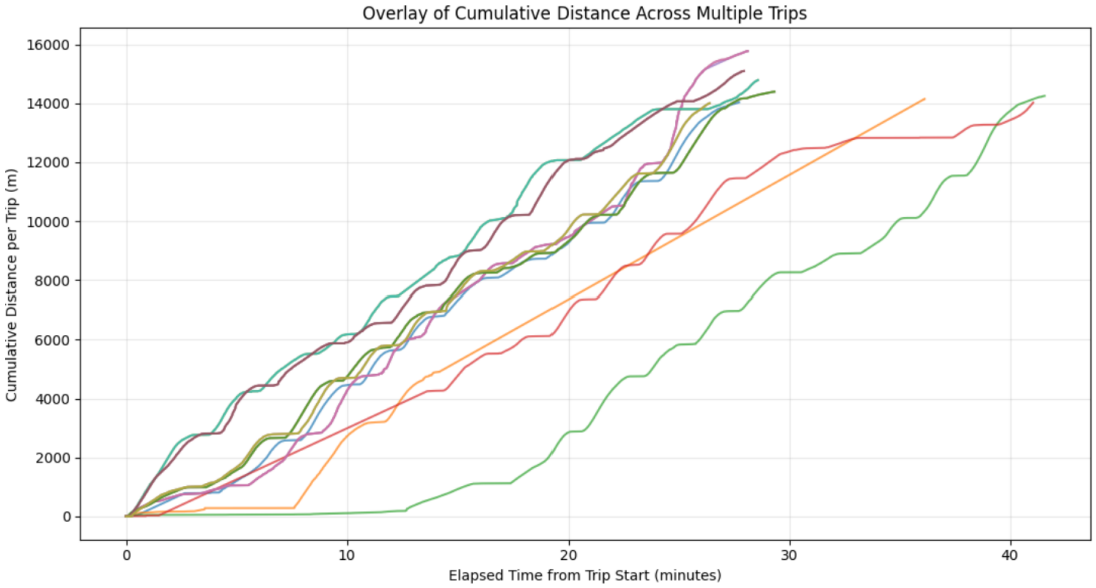


Figure 5: Overlay of cumulative distance across multiple trips, showing the degree of structural alignment between processed journeys. Variation in trip duration and slope reflects differences in train speed and dwell time at stations across sessions, rather than inconsistencies in segmentation.

3.4.1 Signal Features

A *Pearson* correlation heatmap was generated to evaluate relationships between signal metrics. This metric informs on what features exhibit strong or weak correlations amongst other features and the target. Strong positive correlations were observed between all signal metrics, indicating redundancy as seen in Figure 6.

To reduce further noise and improve model efficiency, only a subset of signal features is retained. While *Signal-to-Noise-plus-Interference Ratio (SNIR)* would provide a more comprehensive measure of signal quality, it was not recorded in the collected data. As a result, SNR and *signal quality* are used as the next best alternative.

Additional engineered signal features include:

- **Average signal quality prior to handover for each cell:** For a given cell, this feature evaluates what the signal typically looks like just before the modem does a handover to another cell. This data is inferred from historical data SNR data for each unique cell. This makes it possible for the model to learn the typical, historical SNR threshold for each cell present on the route. When the current SNR approaches that cell's historical average pre-handover SNR, it is a signal that a handover may be imminent.

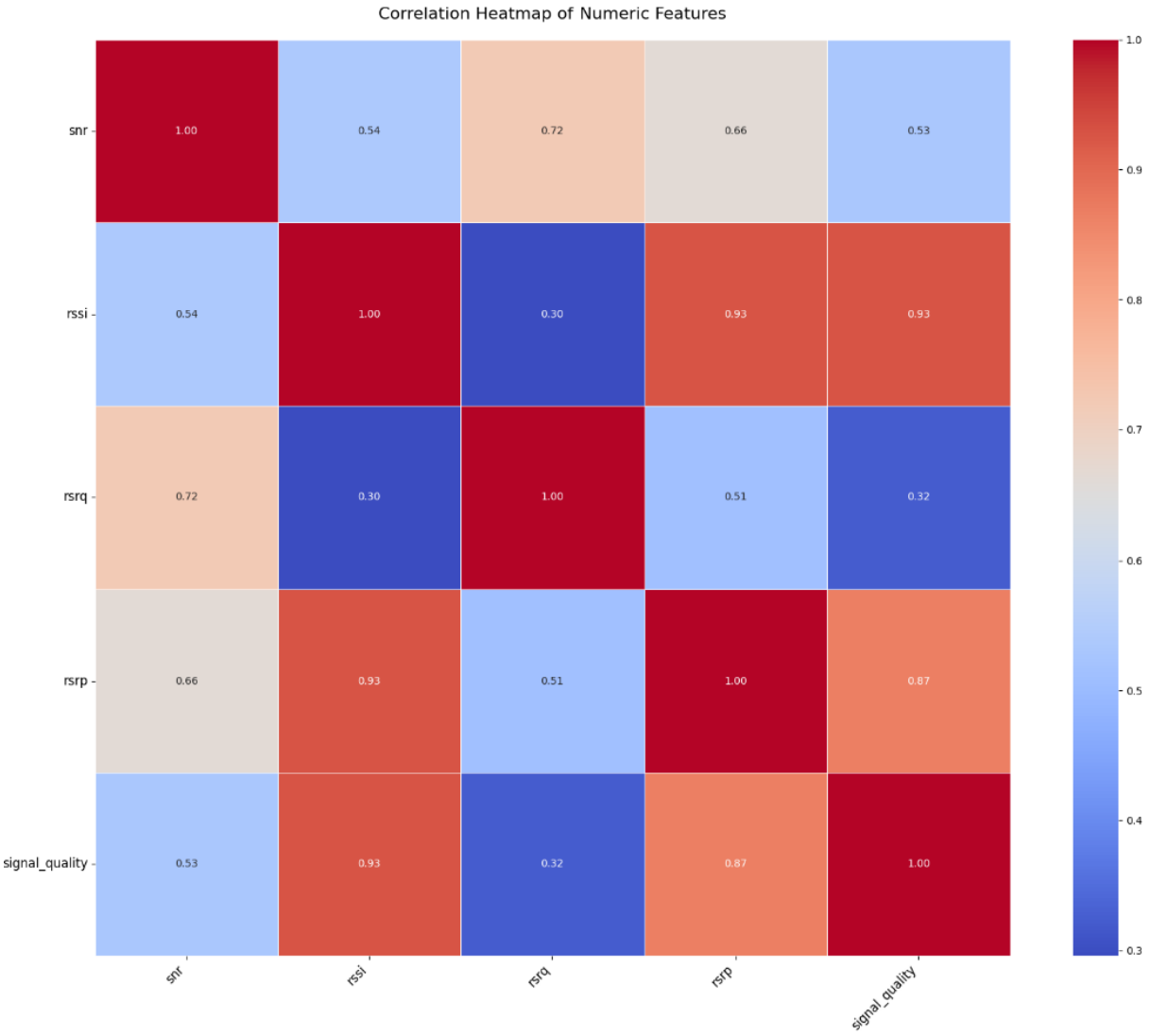


Figure 6: Pearson correlation heatmap of signal features



Figure 7: Handover density measured at each bin along the track showing clustering behaviour. Captured on *Google Earth Pro*

- **Variance of signal quality over a short temporal window:** This feature captures how stable or unstable the signal is in the moments leading up to a handover. An unstable SNR (unstable signal) results in a higher variance. Larger fluctuations of SNR could signal a handover is imminent. These larger fluctuations can be a symptom that the train is entering a cell boundary, where two base stations are competing. Handover can be more likely to occur at these locations. It is important to note that only a limited window of past readings is evaluated. It is futile to look at the entire past context of variance, because these patterns become irrelevant contextually the further they are from the current sample. Typically, the previous 3 or 4 samples are assessed as a minimum to encode variance.

These features aim to capture both steady-state behaviour and instability leading up to a handover.

3.4.2 Spatial Features

Engineered spatial features are derived using cumulative distance, which in itself is also derived and engineered from latitude and longitude as mentioned. The route is partitioned into fixed distance bins (e.g. 200 meter bins), and historical handover density is computed for each bin. The density of each bin is then assigned as a feature to every sample existing in that bin.

This feature captures the likelihood of a handover occurring at a given location based on historical clustering patterns. The exact clustering patterns can be seen in Figure 7. It

is important to note that the denser handover clusters in Figure 7 are consistent visually with tower spacing observed in Figure 9 and Figure 8 as expected.

Table 3: Final feature set used for model training, including brief descriptions and categorisation by feature domain.

Feature	**Category**	**Description**
snr	Signal	Signal-to-Noise Ratio, capturing overall link quality
signal_quality	Signal	Captured using RSRQ
cell_avg_snr_before_handover	Signal (Engineered)	Average signal quality prior to handover for each cell
snr_variance_window	Signal (Engineered)	Variance of signal quality over a 3 sample temporal window
cum_dist_m_per_trip	Spatial (Engineered)	Cumulative distance travelled along the route, computed using the Haversine formula
cell_id	Network	Identifier of the serving cell at each time step
modem_id	Network	Modem that is responsible for sample
speed_mps_gpx	Spatial	Comptutes train speed for current sample
satellites_gpx	Spatial	Indicator of positioning accuracy
delta_dist_m	Spatial (Engineered)	Captures movement dynamics along the route
hist_handover_density	Spatial (Engineered)	Captures location dependent handover likelihood

Additional engineered features were explored during development but were not included in the final model due to limited or no performance gains.

3.5 Model Development Design

More than one problem formulation was explored, with four variations being assessed in total. Over the course of development, the four formulations weren't just alternatives, but they evolved sequentially. Each iteration addressed limitations that were uncovered, refining the predictive approach until satisfactory convergence on the final solution.

These included both regression and several classification-based approaches. Classification-based formulations became the primary focus due to their practical relevance in real systems.

These include:

- Time-to-handover regression
- Binary classification within a fixed future window
- Next-sample binary classification
- Tolerance-based classification

In particular, the tolerance-based formulation is adopted as the final approach, as it strikes the optimal balance between predictive accuracy and practical utility. Predictions made by this classifier are considered correct if they occur within a small, fixed temporal window prior to the actual handover event.

Training is performed using trip-based cross-validation in order to prevent data leakage and ensure that the model is evaluated on unseen journeys. Cross validation was chosen instead of holdout for it's more robust and efficient usage of a limited sized dataset. In addition, trip-based cross-validation is chosen to reflect a realistic deployment scenario, in which models are trained on historical journey data and evaluated on unseen trips. This approach aligns with other work, which assumes that data from previous trips is available for training, while evaluation is performed on new, unseen journeys [1].

Handover prediction along the Tara Street–Howth DART route presents a fundamental modelling challenge: the signal environment is not homogeneous. The urban sprawl in proximity to Tara Street is characterised by more dense cell configurations and more frequent handovers, while the coastal and suburban segments approaching Howth exhibit more stable, lower-density coverage. Figure 9 and Figure 8 contain a detailed frequency of base stations in both areas. A single neural network must implicitly reconcile these local regimes within shared weights, which can hinder its ability to specialise on any one of them. While a standard neural network could theoretically learn these variations, again, it has no explicit mechanism to separate them. To address this, a Mixture of Experts (MoE) architecture was selected, which involves distinct sub-networks (experts) to naturally specialise on different parts of the problem, with the model itself learning when to rely on each expert. Experts operate in parallel, each developing specialised behaviour for a distinct subset of the input space. Although it is difficult to unravel the precise input domain each expert specialises in, one idea is that each expert should focus on an individual part of the track, although there is other possibilities. Furthermore, a separate gating network ingests the same input space as the experts, and produces a probability distribution over the experts; This distribution is used to compute a weighted combination of their outputs. Crucially, the gating weights are computed dynamically at at each prediction, meaning the model can adapt its reasoning strategy continuously rather than applying a fixed function uniformly.

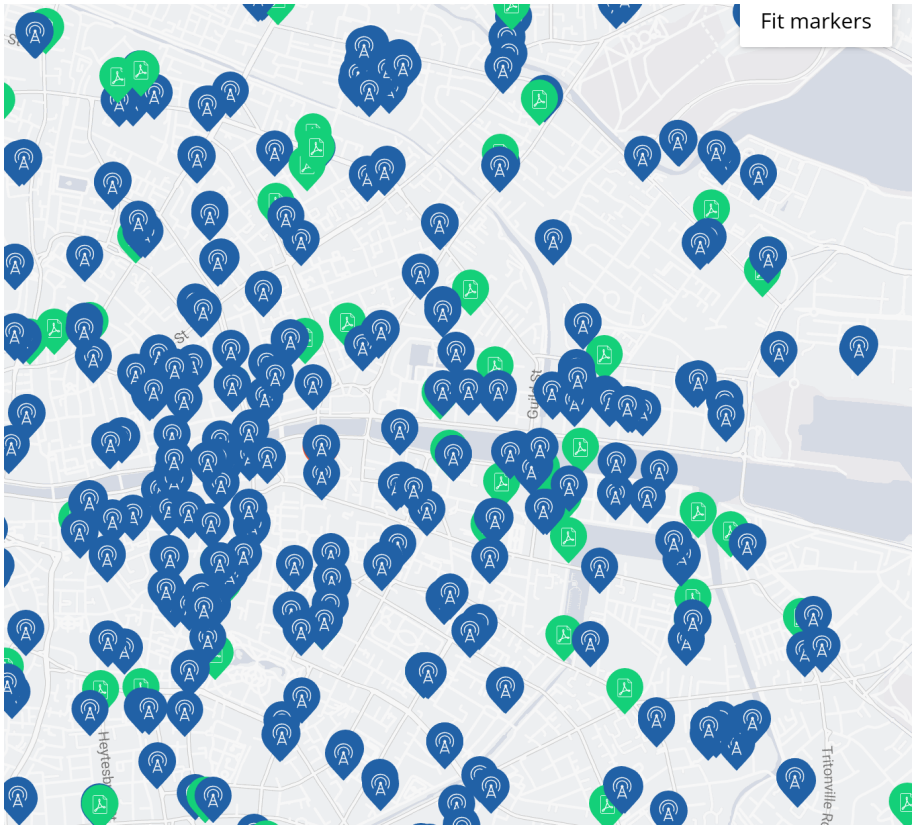


Figure 8: High base station density in Tara Street area, illustrating the dense urban deployment environment in which more frequent handovers are expected. Adapted from [9]



Figure 9: Low base station density in Howth, illustrating the sparse urban deployment environment in which less frequent handovers are expected. Adapted from [9]

Numeric input features capture the signal and mobility context at each sample. This set of features includes signal quality indicators, speed and distance, and historical handover density along the route. Contrarily, categorical identifiers (cell ID and modem ID) are represented using learned embedding vectors rather than one-hot encoding. The motivation for this is to allow the model to discover latent similarity (hidden patterns, i.e. certain cells might behave similarly based on location or network conditions) between cells and between modems. These embeddings are concatenated with the numeric features to form a unified input vector fed to both the experts and the gating network alike.

Each expert is a smaller network designed to produce a scalar output, representing the predicted handover probability. In comparison, the gating network is a more minimal *feedforward* network that maps the same input vector to a normalised (using *softmax*) weight vector over all experts. The result of this setup is that every expert is consulted for every sample, but their contributions are ranked by learned relevance weights. During training, experts are expected to co-ordinate around distinct input regimes as mentioned. An alternative organisation of experts to the aforementioned, maybe that one expert may specialise on high-speed, segments while another focuses on station dwell behaviour. The number of experts is a hyperparameter; tuning here converges on whatever configuration yields best results.

The output of the MoE is the weighted sum of all experts processed through a sigmoid function to produce a final handover probability. Rather than limiting a model to just average across all regimes, MoE allows a model to route each continuous sample to the

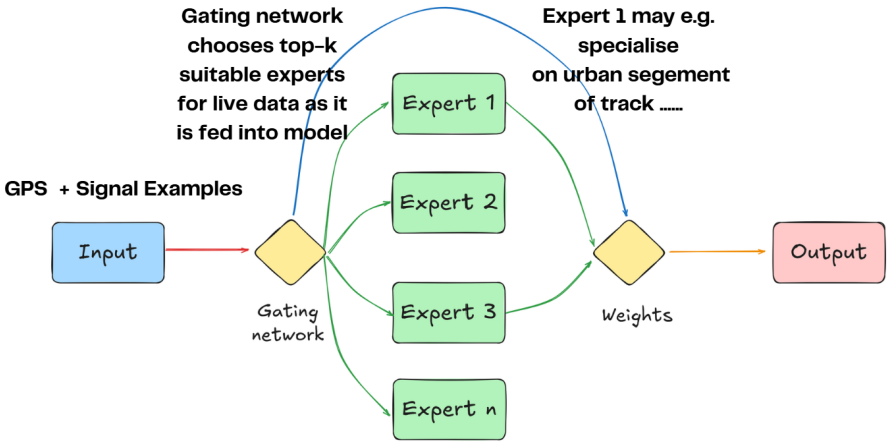


Figure 10: Mixture of Experts architecture for handover prediction. GPS and signal inputs are routed by a gating network across n parallel expert networks, each potentially specialising on distinct regimes along the route. Expert outputs are combined via learned weights to produce a final prediction. Adapted from [8]

experts most suited to its context. Observe a MoE diagram in Figure 10

3.6 Model Evaluation Design

Each problem formulation varied in evaluation metrics. However in all cases, in order to optimally evaluate the models performance, evaluation metrics that provided the most comprehensive context were chosen. For instance, the best and final model (tolerance based classification) prioritised recall, as missing a handover event (false negative) is significantly more costly than generating false positives. Additional metrics such as event detection rate, and subsequently miss rate, are used to provide a more comprehensive and contextual evaluation.

Note that *PR-AUC* assesses class imbalance handling by focusing on how well the model identifies the minority class (handover events), making it an honest measure of performance in this imbalanced setting. Also note that *SMAPE (Symmetric Mean Absolute Percentage Error)* was used as the primary evaluation metric for regression based problem (time_to_next_handover). SMAPE is used alongside MAE (Mean absolute error) to provide a scaled, relative measure of error. While MAE captures absolute deviation, SMAPE expresses error relative to the true value, offering clearer context when evaluating performance across varying time-to-handover ranges.

Table 4: Evaluation metrics used to assess classification model performance.

Metric	**Description**
Precision	Proportion of predicted handovers that are correct
Recall	Proportion of actual handovers that are correctly detected
F1 Score	Balance of precision and recall
Event Detection Rate	Proportion of handover events detected within a predefined tolerance window
Miss Rate	Fraction of handover events that are not detecte
PR-AUC (Precision-Recall Area Under the Curve)	Summarises the precision-recall curve into a single number between 0 and 1, where a higher value indicates better performance.

By integrating spatial and signal features within a robust processing pipeline, the system is capable of capturing the mechanisms of handover behaviour in a repeatable mobility environment.

4 Implementation

4.1 System Overview

At a high level overview, the implemented system is a complete end-to-end pipeline that transforms raw merged LTE modem logs and GPS data into features. These features are then used to train and evaluate predictive model(s) for handover prediction.

The system composes of six primary stages:

1. Data ingestion and preprocessing
2. Data cleaning
3. Data exploration
4. Feature engineering
5. Model training
6. Model evaluation

These stages were all written in *Python* across multiple *Jupyter notebooks*. Python was chosen not as arbitrary choice, but because Python has a vast number of well-maintained libraries for machine learning, data manipulation, and even geospatial analysis. These libraries were integral to enable the development of this model pipeline. Choosing this language meant the wheel was reinvented, allowing time to be invested in the most critical aspects of this project. Python's readability and ease of integration with tools like Jupyter made iterative development and analysis faster than alternatives. In short, Python was chosen because it directly boosted productivity.

Popular Python libraries including *Pandas*, *NumPy*, and *Scikit-learn* were used consistently throughout the pipeline. *Pandas* formed the backbone of the data processing workflow, handling data ingestion, cleaning, merging, and transformation through structured DataFrame operations. *NumPy* supported efficient numerical computation, particularly for array-based operations and feature calculations. *Scikit-learn* was used for standardisation and preprocessing tasks, such as applying `StandardScaler` without data leakage.

The elaborate implementation of each stage is detailed in Figure 11 and also is included in the subsections to follow.

4.2 Data Collection Implementation

LTE modem logs were recorded in *CSV* format, capturing signal metrics at approximately one-second intervals. As mentioned, these logs included RSRP, RSRQ, SNR, cell ID, tracking area code, and additional modem-specific identifiers.

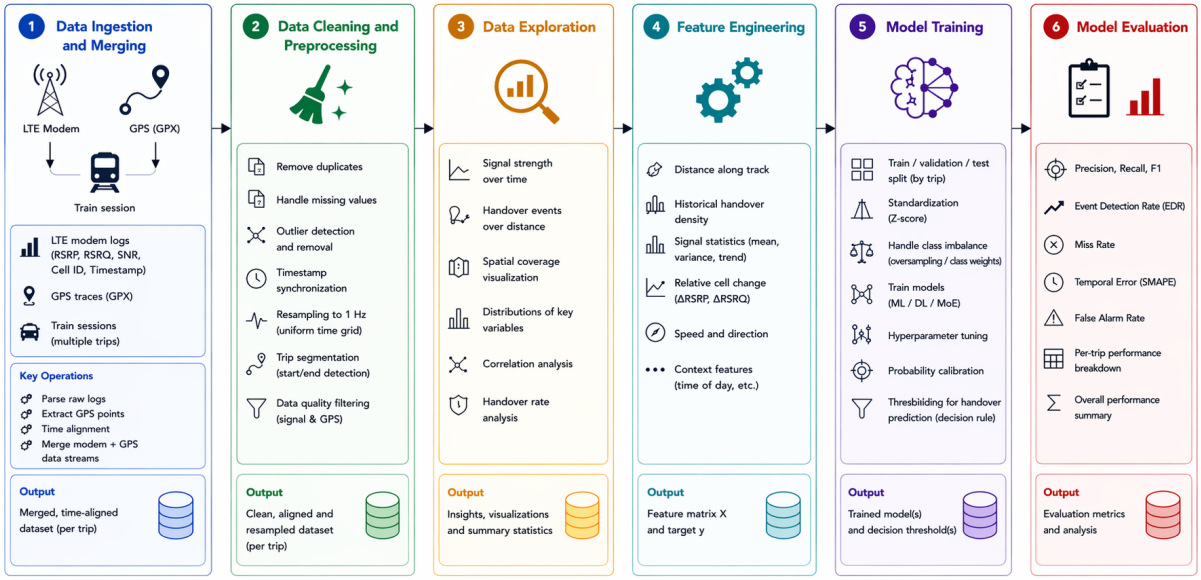


Figure 11: Full implementation pipeline showing raw inputs to final predictions

GPS data was recorded separately on a mobile phone using *BasicAirData GPS Logger for Android*, and exported in GPX format. Each GPS entry contains timestamped latitude and longitude coordinates. The use of two independent logging systems introduced the need for careful alignment between samples of each source during preprocessing.

Multiple data collection sessions were conducted along the Tara Street to Howth route. Later sessions incorporated three modems instead of one, which operated in parallel across different network providers. Each modem sampling was treated independently throughout the model development pipeline, allowing the system to capture variation in behaviour across operators.

To improve segmentation accuracy, manual timestamps for station arrivals and departures were recorded during later collection sessions. These were used as reference points during preprocessing and validation.

4.3 Data Ingestion and Preprocessing

The data preprocessing pipeline was implemented as a reusable, stand-alone merge Jupyter notebook script. The motivation for writing this script, independent of the rest of the model pipeline, was so that each time new data is collected, an automated script ingests and transforms raw CSV and GPX files into a clean, structured dataset. This modular approach is superior to manually adding new data to the merge process, after each new data collection.

GPS and modem data samples were aligned using nearest-timestamp mapping with

a tolerance of one second. For each modem sample, the closest GPS coordinate within this window was assigned. Any unmatched samples occurred typically during brief GPS outages. These samples were excluded from spatial feature computation.

As described in design, handover events were labelled based on changes in cell ID between consecutive samples. This was a simplistic approach providing a consistent definition of handover events across the dataset. Each modem sampling was processed independently to ensure that handovers were correctly attributed to their corresponding network exclusively.

Throughout the pipeline, raw input files are treated as immutable. All transformations and cleaning operations write exclusively to separate output files, ensuring that the original collected data is preserved and any stage of the pipeline can be rerun or debugged without risk of corrupting the source material.

4.3.1 Trip Segmentation

Trip segmentation especially involved multiple improvements until converging on an optimal system. Initial approaches utilised thresholds including speed and dwell-time. However, these methods proved unreliable due to inconsistent GPS sampling and noise. In particular, speed measurements can be more sparse in some trips than others, and often failed to accurately capture stationary periods. This often led to incorrect segmentation. A better approach uses a proximity-based approach. Known coordinates (derived from the data itself and validated using *Open Street Map*) of start and destination stations are used in defining spatial boundaries. A trip is considered to begin when the train enters a defined radius around a start station and ends upon arrival at the opposite terminal. The boundary threshold was set to *100m*, a value that strikes the best balance between accuracy and detection reliability. Instead of using the exact co-ordinates of each station as boundaries, a radius from the station is used to account for varying GPS sampling rates and drift. The final approach proved significantly more effective and prevented compounding segmentation errors observed in earlier implementations.

4.4 Data cleaning

Data cleaning was carried out across two stages of the pipeline. An initial set of cleaning operations was applied during data ingestion and merging, prior to trip segmentation. A second, more targeted set of operations was applied after segmentation, once the dataset had been structured into labelled trip instances. Together, these steps addressed the range of sanitisation issues identified during preprocessing and exploration.

4.4.1 Ingestion-Stage Cleaning

The first cleaning stage was performed during the merging of raw CSV modem logs and GPX data. At this point the data was unstructured, so the desire was to produce a well-formed, consistent dataset in preparation for trip-level processing.

Prior to loading each CSV file, the first example of each file was inspected to determine whether a valid header row was present. Some sporadic files were missing headers, which was observed to be a consequence of occasional logging interruptions observed during data collection. All files were read with `encoding_errors='ignore'` to mutely discard any unreadable bytes introduced by file corruption.

After loading in all data sources, rows that were entirely empty were dropped using `dropna(how='all')`. The `timestamp` column was then parsed with `errors='coerce'`, converting any malformed entries to `NaT`. Any resulting null timestamps were subsequently dropped, as samples without a valid timestamp cannot be aligned to GPS data, rendering them untrainable for the model.

For GPS trackpoints with missing or unparsable timestamps, the removal process was the same as for missing CSV data. The processed remaining GPS data was then used to define a valid recording window. Signal samples falling outside this window (sampling before the GPS logging begins or after it ends) were excluded from the merge, as no reliable spatial context is available for them.

4.4.2 Post-Segmentation Cleaning

After trip segmentation was complete and the dataset had been compartmentalised into labelled journeys, a further sequence of cleaning was applied. This stage focused on reducing feature redundancy, and resolving issues and anomalies identified during exploration.

Trip segmentation identified several sampling periods that did not express a valid train journey. The most frequent anomaly was walking periods between consecutive trips, typically after getting off at the DART, before waiting to embark on the next trip. Focused sanitisation on dwell time at stations, and brief recording intervals before departure enforced that only legitimate journeys are to be included in the final dataset. Specifically, these segments were removed by applying minimum and maximum thresholds on trip distance. A segment was retained only if its cumulative distance exceeded the `length of the track - proximity threshold`. These bounds were derived from the computed (and again validated in *OpenStreetMap*) characteristics of the Tara Street to Howth route. Segments outside these bounds were dropped in their entirety.

Several features were identified during exploration as either redundant or as potential data leakage, and so were removed prior to model training. Some raw GPS columns that were sampled by the modems themselves; `latitude`, `longitude`, `altitude`, and `speed` were dropped. These features had been superseded by the more dense and accurate GPX sampled equivalents. Some features were found to be entirely null across the

dataset, including `earfcn` and `pingpong`. These were also removed. As mentioned in earlier in *Design* rationale, signal metrics exhibiting strong correlation with the retained SNR feature were dropped to reduce redundancy and improve model efficiency. Finally, identifiers that constitute data leakage; `run_id`, `trip_id` and `source_file` were excluded. Temporary features created during preprocessing, and not intended for training were also dropped to reduce redundancy.

In addition, examples where any of the core modelling features, most importantly `cum_dist_m`, that were null, were removed. These columns form the basis of the spatial and signal feature representations used by the model. A sample missing these values cannot contribute meaningfully to training.

All string-typed columns were stripped of leading and trailing whitespace. In addition, a *regex* pass was applied to replace any whitespace-only string values with `NaN`, ensuring that blank fields were consistently treated as missing rather than as valid string entries.

Any remaining null values in the numeric feature set were imputed with zero prior to model input construction. This step was applied after all other cleaning operations to ensure the model received a complete numeric matrix, while avoiding imputation of values that would otherwise be used in feature engineering computations.

4.5 Data exploration

The goal of data exploration is to analyse patterns, anomalies, and relationships within the raw data. These insights inform data cleaning, feature engineering and model design. By visualizing relationships among features, we identify both expected and unexpected behaviours and outliers. Rigorous data exploration ultimately guides decisions for further stages.

Beyond the earlier data cleaning analysis, a useful application of data exploration was to evaluate the per-trip handover count and direction analysis. Using a Haversine-based function, each trip is classified either eastbound (Tara $\rightarrow$ Howth) or westbound (Howth $\rightarrow$ Tara), along with handover counts per trip. Although direction was originally computed to be a threshold in trip segmentation, questioning whether handover frequency differs by direction was also a feature candidate. Although the physical track for each direction is almost identical, the train's traversal behaviour could vary considerably due to other factors such as speed and dwell time at stations. Aggregating handover counts per trip outlined how consistent or variable the network behaviour is across multiple journeys of the same track.

Handover frequency varies slightly across modems, with *Modem 1* exhibiting the highest rate at 6.83 percent, compared to 4.08 percent for *Modem 2* and 3.06 percent for *Modem 0*. These findings suggest that differences in network behaviour across operators, potentially due to varying cell density, coverage strategies, or handover policies. Outputs show that 29,223 out of 30,541 rows did not have a handover, which is approximately a

96/4 split. This class imbalance is expected as the same imbalance ratio exists in the dataset utilised in [**?**]. This imbalance is severe enough that it directly shapes further model choices.

MatPlotLib and *Seaborn* was used in creating the 2D figures during data exploration and beyond. Evidence of these two libraries' usage include Figure 6, Figure 5 and Figure 14. *Google Earth Pro* was utilised for more textured, 3D schematics, with examples including Figure 7 and Figure 4

4.6 Feature Engineering

Feature engineering was implemented within the data pipeline, following data exploration, and before model training. `Cumulative distance` was computed earlier in the pipeline than other engineered features, due to its necessity in aligning spatial context, and being utilised for both data exploration and subsequent feature construction. `Cumulative distance` was computed by summing incremental distances between consecutive GPS coordinates using the Haversine formula as mentioned. *Geopy* was used to compute geodesic distances between consecutive GPS coordinates. An important implementation detail was ensuring that cumulative distance resets at the beginning of each trip, rather than tallying a total count across consecutive trips. Early versions of the pipeline incorrectly accumulated distance across multiple trips, leading to inconsistent spatial representations. By resetting distance per trip and per modem, each journey could be aligned and contrasted together, along a consistent spatial axis. This enabled more meaningful usage of historical behaviour across the entire pipeline including training.

Three additional features described earlier in *Design*, were engineered to provide the model with richer signal and spatial context beyond the raw measurements. `cell_avg_snr_before_handc` was implemented by iterating over every handover event in the dataset, grouped by modem and trip. For each event, a three-second look-back window is applied to assess SNR samples recorded on the same serving cell immediately prior to the handover. These samples are averaged per cell across all observed handover instances, producing a per-cell historical SNR threshold. This lookup table is then merged back onto the full dataset by `cell_id`, so that each row is annotated with the average pre-handover SNR of its current serving cell. The key implementation detail is that this feature is constructed solely from handover events, meaning it encodes the specific signal conditions under which each cell on the route has historically initiated a handover, rather than its general average signal level.

`snr_variance_window` was implemented as a rolling variance over a window of four consecutive samples, applied independently per modem and trip after sorting chronologically. Grouping by `modem_id` and `trip_id` before applying the rolling operation was a critical implementation detail, ensuring that variance does not bleed across trip or mo-

dem boundaries and conflate signal behaviour from separate journeys. A minimum period of three samples is enforced before a variance value is computed, preventing misleading near-zero values at the start of each trip where insufficient history exists.

`hist_handover_density` was implemented by first compartmentalising `cum_dist_m_per_trip` into fixed 200-metre bins, assigning each row a `distance_bin` label. Handover counts and total sample counts are then aggregated per bin across all trips, and the ratio of the two is stored as the density value for that bin. This bin-level lookup is merged back onto the dataset, annotating every sample with the historical handover probability at its corresponding position along the route. The 200-metre bin size was chosen to provide spatial grouping granular enough to capture localised clustering behaviour, while remaining coarse enough that each bin is populated by a meaningful number of samples across trips.

As described in each implementation, all engineered features were computed using only past information, avoiding any leakage of future data into the model.

4.7 Model Training and Development Implementation

Model development was implemented using *PyTorch*, structured around the tolerance-based binary classification formulation selected during design. The implementation progressed through four problem formulations before converging on this approach, each of which required distinct target construction, loss functions, and evaluation strategies.

4.7.1 Problem Formulation

Four formulations also outlined in *Objective and its Context* were implemented and evaluated. While all four problem formulations were explored earlier, the focus here is solely on the implementation of the final chosen approach, as it formed the basis of the conclusive evaluation and results. The tolerance-based target was constructed using a dedicated labelling function applied after all feature engineering. For each handover event identified by `handover_next_sample`, a window of $k = 5$ samples including and before the actual event is marked as positive. The result is stored in a new column `handover_window`, which serves as the training target. This formulation acknowledges that predictions made slightly before the precise handover timestamp are still operationally useful, and avoids penalising the model for near-misses. This labelling strategy is illustrated in Figure 12

4.7.2 Model Architecture

The model was implemented in PyTorch using a Mixture of Experts architecture. This approach follows practical implementation patterns described in [10]. The MoE architecture described in *Design* was implemented as a single `nn.Module` in PyTorch using

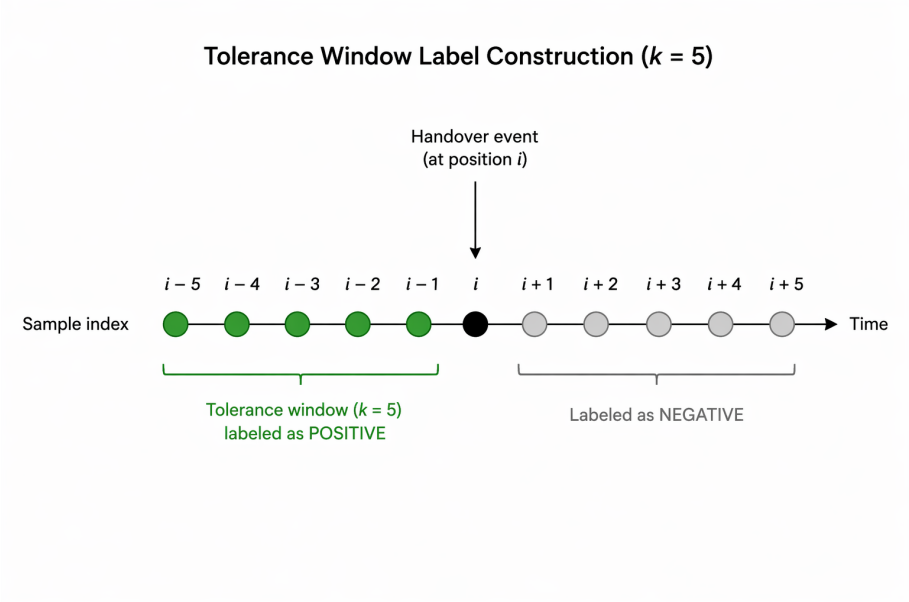


Figure 12: Tolerance-based label construction for $k = 5$. Only samples preceding the handover event are labelled as positive.

several hyper parameters. Categorical features such as `cell_id` and `modem_id` are represented using learned embedding layers of chosen dimension 32 each. These embeddings are looked up for each sample and concatenated with the z-score normalised numeric feature vector to form a unified input representation. The total input dimensionality is therefore: `the number of numeric features plus the number of nominal features` $\times$ `the number of embedding dimensions.`

Five expert networks are instantiated as parallel `nn.Sequential` modules. Each expert is a three-layer feedforward network with hidden dimensions of 256 and 128, separated by `ReLU` activations, and producing a single scalar output. The gating network is a shallower two-layer feedforward network with a hidden dimension of 64, whose output is passed through `Softmax` to produce a normalised weight distribution over the five experts. Both the experts and the gating network receive the same concatenated input vector. The final model output is computed as a weighted sum of all expert outputs, producing a single scalar per sample which is passed through `Sigmoid` at inference time to yield a handover probability.

4.7.3 Training Procedure

Training used `BCEWithLogitsLoss` as the loss function, which combines a sigmoid activation with binary cross-entropy in a numerically stable formulation. The *Adam* optimiser

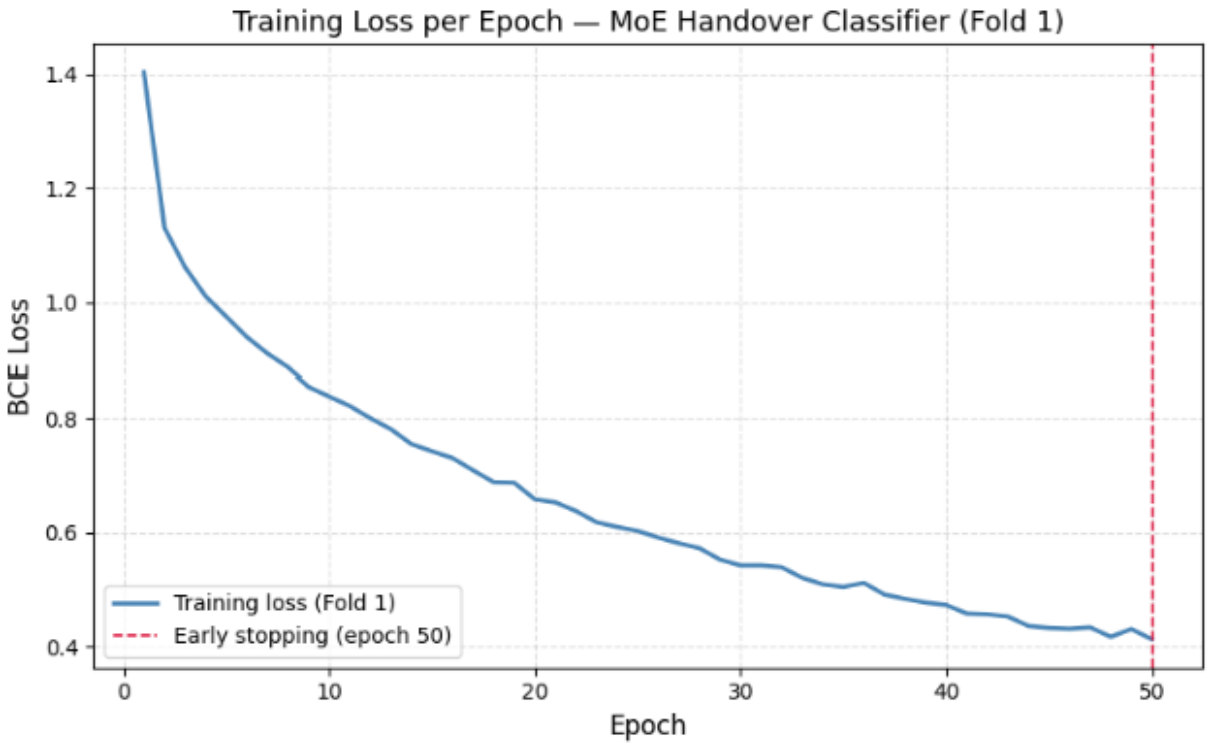


Figure 13: Training loss per epoch for the MoE handover classifier on a representative fold (Fold 1). The curve demonstrates rapid initial convergence followed by diminishing improvement beyond approximately epoch 35, supporting the choice of 50 epochs as a sufficient training number. The dashed line indicates the early stopping boundary.

was used with a learning rate of 0.001, paired with a dynamic `ReduceLROnPlateau` scheduler that halves the learning rate after 3 epochs without improvement in validation loss. A dynamic learning rate aided the model in converging at the optimal learning rate automatically. Training ran for a maximum of 50 epochs per fold with a batch size of 256. Model performance was observed to plateau beyond the chosen number of epochs as seen in Figure 13, with increasingly diminishing improvements in validation metrics.

Resampling was achieved by applying a `pos_weight` to the loss to compensate for the 96/4 class imbalance. Removing explicit class weighting and instead relying on the tolerance-based label expansion increases the effective proportion of positive samples. This yielded more stable and intelligently calibrated outputs. The `pos_weight` parameter was retained in the implementation but capped at a maximum value of 10 to prevent extreme gradient scaling in folds with particularly severe imbalance.

4.7.4 Cross-Validation Strategy

Cross-validation was implemented as a *leave-one-instance-out* scheme over unique `trip_instance` identifiers, where each instance corresponds to a single modem recording of a single trip. With 19 trip instances in total, this produces 19 folds, each holding out one instance for evaluation, while training on the remaining 18. Instance identifiers were shuffled pseudorandomly prior to fold assignment to avoid any ordering bias. Within each fold, the `StandardScaler` is fit exclusively on the training partition and applied to the test partition, ensuring that feature scaling does not leak any distributional information from the held-out trip.

4.7.5 Threshold Selection

Model outputs are continuous normalised probabilities, requiring a fixed decision threshold for binary classification. Given the priority of recall in the final model, missing a handover event is significantly more costly than generating a false positive, so a low threshold of 0.01 was selected. This was determined empirically by evaluating precision and recall across a range of threshold values on aggregated predictions across all folds. The precision-recall curve was used as the primary diagnostic, with PR-AUC serving as the threshold-independent summary metric. As illustrated in Figure 14, a threshold of $t = 0.01$ was selected to maximise recall, accepting a reduction in precision as an acknowledged trade-off.

4.8 Model evaluation Implementation

Model evaluation was performed by aggregating predictions across all 19 folds into a single group of predicted probabilities. Evaluation metrics were then computed over this aggregated output rather than averaged per fold, ensuring that performance on smaller trip instances was not over-weighted. Metrics computed include precision, recall, F1 score, and PR-AUC as the primary threshold-independent summary. PR-AUC was prioritised over ROC-AUC given the severe class imbalance, as it is more sensitive to performance on the minority class. An event detection rate was additionally computed to assess what proportion of true handover events were captured within the tolerance window, providing a more operationally meaningful evaluation in comparison to sample-level recall.

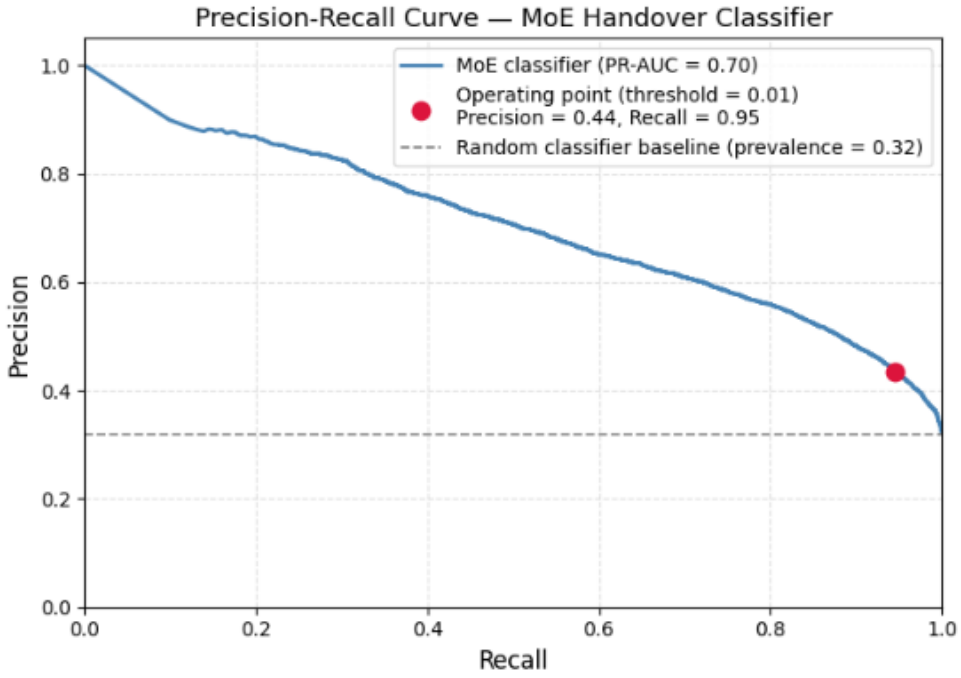


Figure 14: Precision-recall curve for the MoE handover classifier, aggregated across all 19 cross-validation folds. The red marker indicates the chosen operating point at threshold $t = 0.01$, yielding precision 0.44 and recall 0.95. The dashed baseline represents a random classifier at the dataset's positive class prevalence.

5 Evaluation

5.1 Experimental Setup

The evaluation of the implemented system is engineered to assess the effectiveness of predicting handover events under practical conditions.

As described in the *Design* and *Implementation* sections, evaluation is conducted using a leave-one-instance-out cross-validation strategy, where each instance corresponds to a single modems recording of a single trip. By enforcing trip separation, the model is required to understand underlying spatial and signal patterns, instead of memorising recorded journeys.

Model performance is assessed using a combination of standard classification metrics, but also domain-specific metrics particularly suited to this handover prediction problem:

To reiterate, given the application context, recall and detection rate are prioritised for the evolved, final model. Missing a handover event (false negative) directly impacts Quality of Service, and by contrast, false positives only introduce more conservative mitigation behaviour. This aligns with prior work [1], which highlights the critical importance of minimising missed events.

5.2 Problem Formulation Comparison

Four problem formulations as outlined in *Analysis* were implemented and evaluated, reflecting an iterative refinement process rather than independent alternatives. While the most robust model is clearly evident through evaluation, the less successful formulations provide worthwhile insight into the limitations of alternative approaches, justifying their exclusion.

5.2.1 Time-to-Handover Regression

The regression formulation attempts to predict the time remaining until the next handover event. While conceptually appealing due to its fine-grained output, this approach proved highly unstable in reality.

Error analysis shows that predictions near the handover boundary are highly sensitive, where small deviations result in large relative errors. This is reflected in SMAPE values in the 0–10 second range as seen in Figure 15. This model would perform especially poorly in more urbanely sprawled rail environments, where infrastructure is likely to enforce a rapid handover frequency. While SMAPE values decrease at the 10–60 second range, variance increases for larger prediction horizons due to the uncertainty of future network conditions. Overall however, Figure 15 reflects instability in regression-based predictions.

Overall, this formulation was deemed unsuitable for reliable real-time prediction.

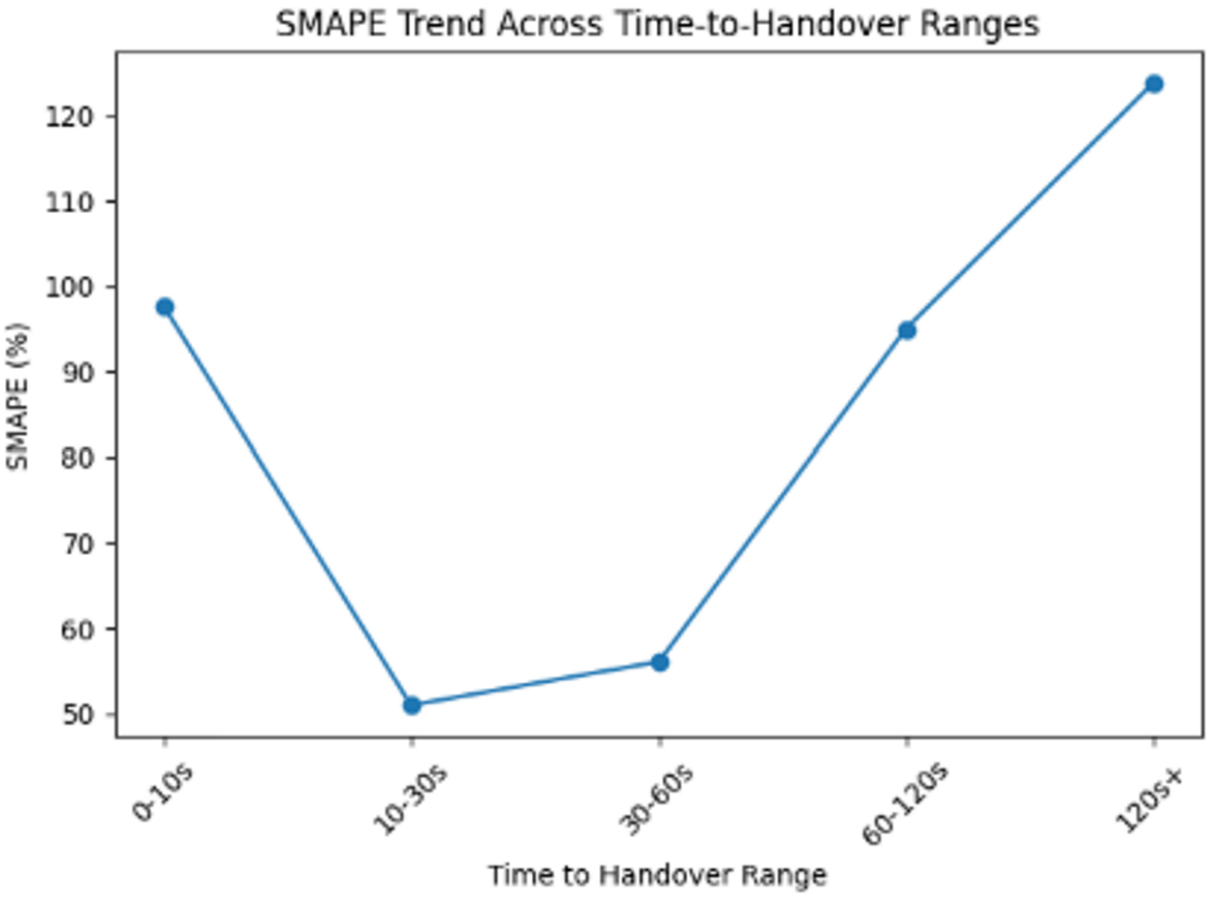


Figure 15: SMAPE across time-to-handover ranges

5.2.2 Binary Classification (Next 1 Second)

This formulation predicts whether a handover will occur within the next second. While this aligns quite well with real-time system requirements, it results in a highly imbalanced dataset with a large proportion of positive samples (*approximately 70%*).

In practice, this leads to highly inflated recall but poor discriminative power, as naive strategies (predicting frequent positives) achieve competitive performance upon first glance. The dummy classifier alone achieves 0.7 recall. As a result, this formulation lacks practical utility despite superficially strong metrics.

5.2.3 Binary Classification (Next Sample)

Predicting handover occurrence at the next sample introduces a stricter and more balanced formulation *(approximately 4% positives)*. However, this significantly increases prediction difficulty.

The model struggled to learn meaningful patterns under this constraint, with performance only marginally exceeding baseline levels. This highlights the challenge of predicting exact handover timing in this context.

5.2.4 Tolerance-Based Classification

To relax the target outlined in `next_sample`, while preserving operational value, a tolerance-based formulation was introduced. In this approach, predictions are considered correct if they occur within a window of k samples prior to a handover event.

This formulation aligns closely with practical system requirements, where early warning is sufficient and exact timing is not required. It also improves learnability by allowing the model to capture pre-handover patterns rather than a single precise point.

The model achieves an event detection rate of 99%, detecting at least one prediction per true handover event in nearly all cases. Recall is 94%, indicating that while individual positive samples are correctly identified with high frequency, a small fraction of samples within handover windows remain undetected. The gap between event-level and sample-level performance is expected and acceptable. Missing isolated samples within a detected event (False negatives) is certainly less critical than missing an entire event.

This formulation was selected as the final approach. Results for all four formulations our outlined in Figure 5

5.3 Tolerance Window Analysis

The impact of tolerance window size was evaluated across multiple configurations.

Training and evaluation with $k = 2$ demonstrated a stricter target closer to `next_sample`, achieving strong detection performance while maintaining reasonable precision. Increasing the evaluation window to $k = 3$, resulted in slight improvements, indicating that

Formulation	SMAPE	F1	Precision	Recall	PR-AUC
Time-to-Handover	75.4 percent	—	—	—	—
Binary (Next 1s)	—	0.91	0.88	0.99	—
Binary (Next Sample)	—	0.19	0.12	0.44	0.11
Binary with Tolerance (k)	—	0.6	0.45	0.94	0.7

Table 5: Comparative performance across formulations

while an increase in tolerance improves performance, it comes at the cost of operational value. $k = 5$ with a threshold configured for maximum event detection rate and recall demonstrates the best performance, but is possibly the upper bound of target leniency. Full performance results for each tolerance window are illustrated in Figure 6

Tolerance Window	Detection Rate	F1	Precision	Recall	PR-AUC
$k = 2$	—	0.5	0.4	0.66	0.5
$k = 3$	—	0.57	0.47	0.71	0.57
$k = 5$	0.99	0.6	0.45	0.94	0.7

Table 6: Comparative performance across different tolerance windows

5.4 Model Performance

The final Mixture of Experts tolerance based classifier achieves consistently strong performance across all cross-validation folds. Although per-fold metrics are omitted for brevity, evaluation across all folds captures variability between journey instances. This variation reflects the diversity of historical trips and strengthens the robustness of the reported performance. Outlined in Figure 7 is the full analysis. Recall and event detection rate remain relatively stable throughout every fold.

Table 7: Per-fold performance variation across 19 cross-validation folds.

Metric	Mean	Std Dev	Min	Max
Precision	0.44	0.07	0.31	0.56
Recall	0.94	0.03	0.86	0.99
F1 Score	0.60	0.07	0.45	0.70
Event Detection Rate	0.99	0.02	0.93	1.0

These results demonstrate that the model reliably detects the vast majority of handover events. High recall is achieved through the use of a deliberately low decision threshold ($t = 0.01$), which biases the model towards detecting positives.

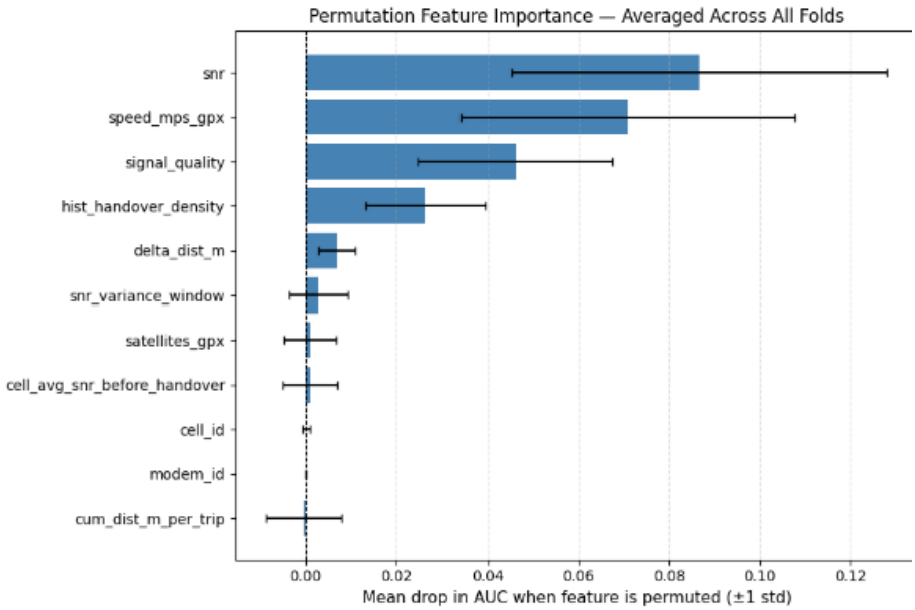


Figure 16: Permutation feature importance averaged across all folds, expressing the contribution of each feature to model performance (measured by decrease in PR-AUC). The model is primarily driven by signal quality and mobility features, with minimal reliance on identifier based inputs. The error bars will also demonstrate that feature importance experiences variability across trips.

5.5 Threshold Sensitivity

Performance is highly sensitive to the decision threshold applied to model outputs. Lower thresholds significantly increase recall and detection rate but reduce precision. Conversely, higher thresholds improve precision at the cost of an increased miss rate (more False Negatives). Each iteration of training and evaluation on `next_1_second` notably always achieved merely a threshold approximating 0.50. This means the model is no better than flipping a coin, because it isn't picking up meaningful patterns from within the data.

In the tolerance based formulation, the model used threshold clustering close to 0.0 or 1.0, indicating it did identify a pattern. The selected operating point ($t = 0.01$) reflects the prioritisation of recall, consistent with system requirements. This trade-off is visualised in the precision-recall curve illustrated in Figure 14. Importantly, the model consistently outperforms the random baseline (0.32), as reflected by a PR-AUC of 0.70, indicating meaningful discriminative capability across thresholds.

5.6 Feature Contribution Analysis

Feature weight analysis for the threshold based formulation as illustrated in Figure 16, shows that signal quality metrics are the most influential contributors to model performance. In particular, SNR exhibits the highest contribution, with a substantial drop in performance when left out, indicating that the model relies heavily on signal strength information to anticipate handover events.

`speed_mps_gpx` and aggregated signal quality features also exhibit meaningful prediction power. This suggests that both mobility metrics and overall radio conditions play an important role in determining handover likelihood. This aligns with outlined expectations, as rapid movement and degrading signal conditions are known precursors to a change in serving cell.

Route historical context, captured through `hist_handover_density`, contributes moderately, indicating that concentration patterns of past handovers provide useful, but non-paramount predictive signalling.

In contrast, features such as `cell_id`, `modem_id`, and cumulative distance exhibit negligible importance throughput folds. This can be considered a positive observation, as it suggests that the model does not memorize specific cells or routes, but instead generalises based on underlying signal regimes. This indicates robustness to unseen conditions.

Moreover, some features show near-zero or slightly negative importance across folds, implying redundancy or even noise. These features may not contribute meaningfully to predictive performance and should be candidates for removal in future iterations of the model.

Another interesting feature considered was `projected distance in` T `seconds, where` T `represents a future time horizon`. This represents an estimate of future location along the route. Knowing current position on the track from `cum_dist_per_trip`, along with current `speed` , enables projection of the future position the train will be at in T seconds. This projected position could be compared to past historical clustering to evaluate if this location is a historical handover hotspot. This feature introduces circular dependency problem: to project a future position in T seconds, T is required, however T itself is the prediction target here. This creates a feedback loop in which prediction errors can propagate, leading to incorrect spatial projections and degraded performance. Additionally, reliance on current speed for projection introduces further uncertainty, as speed estimates are subject to noise. As a result, this feature was deemed unstable and unsuitable for reliable prediction.

5.7 Expert Utilisation in MoE

Analysis of the Mixture of Experts architecture reveals non-uniform utilisation across experts, indicating that the gating network is not assigning inputs arbitrarily. Mean expert usage values of 0.1818, 0.1727, 0.1916, 0.2150, and 0.2389 demonstrate a moderately imbalanced distribution, while still maintaining active contribution from all experts. The system has not collapsed, meaning that a small subset of experts has not dominated the gating assignments. This suggests every expert is utilised. Distinct experts are activated for different subsets of the data, suggesting specialisation.

This behaviour is consistent with the heterogeneous nature of the railway environment, where signal characteristics differ between dense urban areas and more open coastal regions.

5.8 Key Findings

The evaluation highlights several important findings.

Handover events exhibit repetitive spatial clustering along the route. As shown, handovers consistently occur in historical high density regions across multiple journeys. This further validates that handover behaviour is a learnable pattern, which is determined by underlying persistent infrastructure and spatial conditions.

Secondly, spatial features based on the observed historical clustering contribute a meaningful role in predictive performance. For example, `historical_handover_density` significantly improves model performance, confirming that incorporating route-aware context is worthwhile. Network identifiers such as `cell_id` do not play as much of an important role in prediction as initially anticipated. Signal based features, most notably SNR, are the biggest influence in prediction.

The choice of problem formulation has a major impact on performance. Regression-based approaches proved unstable, particularly near handover boundaries, and for remote future events. The next-sample formulation, while theoretically precise, was too strict and difficult to learn given the constrained dataset. In contrast, tolerance-based classification provided a practical balance between learnability and operational value, allowing the model to capture meaningful pre-handover patterns.

The final model achieves high recall, and near-perfect event detection rate, demonstrating that handover events can be reliably identified pre-emptively. While precision remains moderate, this trade-off is acceptable given the system objective of minimising missed events.

Finally, analysis of the Mixture of Experts architecture shows that all experts are being utilised in training and evaluation. While the degree of specialisation is only moderate, the gating mechanism demonstrates input-dependent routing behaviour.

Overall, these findings confirm that handover prediction can be feasible and reliable when high mobility environments are structured with spatial and signal context effectively.

5.9 Limitations

Despite positive and meaningful results, some limitations remain.

As mentioned earlier, a notable constraint on this work is the limited size dataset . While cross-validation mitigates this shortcoming to some extent, the limited volume restricts the diversity of conditions captured, and may impact the robustness and performance of the model.

Furthermore, another limitation of this work is the class imbalance present in the dataset, where handover events represent a small minority of samples. Although mitigation strategies such as class weighting and tolerance-based label construction were applied to improve learning stability, imbalance can still bias the model towards majority class behaviour. Moreover, performance metrics may be sensitive to this imbalance, reinforcing the need for contextual interpretation of results, particularly in relation to precision and recall trade-offs.

In addition, the dataset is constrained to a single railway route (Tara Street to Howth and back). While this aligns with the design choice of route-specific modelling, it limits generalisability. The model can perform well on this route, but it is unknown if the same model can transfer directly to other environments and exhibit similar performance without retraining on new specialised data.

Operator diversity is also limited, with data collected from only three network carriers. This limitation means the data may not fully capture the variability in handover behaviour across different network configurations.

The tolerance-based formulation, while performing positively, introduces a trade-off between accuracy and practical operational value. Increasing the tolerance window improves recall and detection rate, but reduces resolution in how precise predictions are made, when predicting the exact timing of a handover event. As observed, performance improves up to $k = 5$, but beyond this point the formulation becomes overly lenient and less meaningful operationally.

Additionally, the model prioritises recall through the use of a very low decision threshold ($t = 0.01$). This results in a higher number of false positives (lower precision). While this can be acceptable for the intended application, it may lead to inefficient data loss mitigation behaviour if not carefully managed.

6 Conclusion

6.1 Discussion

To contextualise and review the evaluation, it is best to discuss the key findings and limitations outlined. At a higher level, the results show that while handover behaviour is significantly influenced by spatial location, signal characteristics are the most dominant

factors in prediction. Perspective now shifts from reaction to predictive, location-context aware modelling, where prior knowledge of the environment plays a meaningful role.

With the success of the tolerance-based formulation emerges an important practical insight: precise timing of handovers is noisy and difficult to model directly. However, identifying the conditions leading up to a handover is significantly more achievable and operationally useful. This reframing aligns the modelling approach with real-world system requirements, where early warning can be just as valuable as exact prediction.

From a system design perspective, the prioritisation of recall and event detection rate reflects a deliberate trade-off that favours reliability over efficiency. The model's tendency to over-predict events is not a flaw, but a consequence of the structure of the problem, where missed handovers are more detrimental than false positives. This reinforces the idea that model evaluation must be context-driven rather than purely metric-driven. `handover_next_1s` is a prime example of inflated metrics that look impressive, but have little to no contextual operational value.

Vitally, outlined limitations in *Evaluation* suggest that while the results are promising, further validation is required before deployment in broader contexts.

Overall, the results certainly showcase that effective handover prediction is dependent on overall model complexity, richness of features, and data. However, the importance of relevant and correct problem formulation cannot be understated.

These findings extend beyond experimental validation, demonstrating clear potential for real-world deployment. In multi-link systems such as Peplink, a predictive model of this nature could be integrated to enable proactive link management across multiple cellular interfaces. By anticipating imminent handover events, the system could dynamically reroute or replicate latency-sensitive traffic, reducing service disruption and improving overall Quality of Service.

6.2 Answer to Research Question

The central research question of this project can be stated as follows:

Can handover events in a railway environment be predicted reliably using machine learning models trained on signal, spatial, and temporal data?

Based on the results presented, the answer is mostly yes, with important qualifications.

Handover events can be predicted with high reliability when:

- The model incorporates accurate signal and spatial features derived from historical data,
- The prediction task is framed as a tolerance-based classification problem,
- The system is optimised for high recall and detection rate.

Under these conditions, the model achieves near-perfect detection rates across multiple validation folds, demonstrating that the underlying patterns governing handover behaviour can be learned.

However, exact timing prediction remains challenging, at least under the constraints this work experienced. Performance is dependent on the availability of route-specific historical data. As such, the approach is best suited to persistently structured and repeatable mobility environments.

Notably, it is now possible to contract the scope of the research question: temporal data is redundant if well-formed spatial and signal context are available.

6.3 Contribution

This project builds on the current research and work done on mobility management in cellular networks. Firstly, it demonstrates that handover behaviour in railway environments is predominately correlated with signal , but also with spatial context. As a result, this information can be effectively exploited using machine learning techniques. Secondly, it introduces a route-specific modelling approach that aligns historical data along a cumulative distance representation, enabling consistent comparison across repeated journeys. Next, it provides a systematic evaluation of multiple problem formulations, showing that tolerance-based classification offers a practical balance between predictive accuracy and real-world utility. Finally, it presents a complete end-to-end pipeline, from raw data collection to model evaluation, traversing the challenges and considerations involved in applying machine learning to real-world network data.

6.4 Future Work

Several directions for future work arise from this study.

- **Expanded Data Collection:** Collecting data across additional routes and environments would improve generalisability and diversity of data for learning.
- **Real-Time Deployment:** Integrating the model into a live system for real-time prediction and evaluation.
- **Multi-Link Optimisation:** Applying predictions to drive intelligent traffic management strategies such as selective packet replication.
- **Improved Feature Design:** Incorporating additional features such as network load or neighbouring cell information.

These extensions would further validate the practical applicability of this work.

6.5 Reflection

While this project largely involved the design and implementation of a complete machine learning pipeline, a significant portion of the work was dedicated to data collection, cleaning, and validation. This process proved to be both time consuming and non-trivial, requiring the integration of multiple data sources, alignment of timestamps, and careful handling of missing or inconsistent values. In practice, raw data collected from real-world environments is often noisy, incomplete, or misaligned. Substantial effort was required to ensure consistency, remove invalid samples, and construct reliable features such as cumulative distance and handover labels. This highlights a key insight from the project: that the performance of the model is fundamentally dependent on the quality of the data it trains and is evaluated on. Although discernibly many of the improvements observed throughout development were the result of model refinement, improvements in data preprocessing and feature engineering also contributed significantly to overall performance. Overall, this work reinforces the importance of thoughtful and forensic data handling in applied machine learning, where the majority of effort is often spent ensuring that the input to the model is consistent and meaningful. Additionally, the project highlighted the importance of aligning the problem formulation with practical system requirements, as some technically valid approaches proved ineffective in real-world scenarios. From a technical perspective, the project required the integration of multiple domains, including wireless networking, data engineering, and machine learning. Key skills developed during the process include feature engineering, model evaluation, and experimental design.

In retrospect, additional time spent on early data validation and collection would likely have improved overall outcomes. However, the iterative approach adopted allowed for continuous iterative improvement and provided valuable insights into this particular problem domain.

6.6 Project Timeline

Work for this project began in last October, and was conducted over two academic semesters, with the following high-level phases:

- **Semester 1:** Problem formulation, initial data collection, and exploratory analysis.
- **Early Semester 2:** Data processing pipeline development and feature engineering.
- **Mid Semester 2:** Model development and experimentation with multiple formulations.
- **Late Semester 2:** Evaluation, final refinement, and report writing.

The timeline reflects the iterative nature of the project, with multiple cycles of data refinement and model improvement.

7 Appendix

References

[1] K. Fahmi and D. Leith, "Smart replication for seamless and efficient real-time communication in underground railway trains," in *Proc. 2021 International Conference on Computer Communications and Networks (ICCCN)*, IEEE, 2021.

[2] Parraga-Villamar, V., Guaman, D. S., Grijalva, F., and Lupera-Morillo, P. (2025). A Systematic Literature Mapping of Machine Learning for Handover Optimization in Cellular Networks. IEEE Access, 13, 144699-144732. https://doi.org/10.1109/ACCESS.2025.3599475

[3] J. Geurts, K. De Schepper, S. Latré, and F. De Turck, "Transparent connectivity maps for mobile applications," in *Proc. 13th International Conference on Network and Service Management (CNSM)*, IEEE, 2017, pp. 1–9.

[4] Peplink, "Peplink Multi-WAN Technology," [Online]. Available: `https://www.peplink.com`. [Accessed: Apr. 2026]. https://www.peplink.com/

[5] "Handover Events in LTE," [Online]. Available: `https://www.scribd.com/document/583643654/Handover-Events-in-LTE` [Accessed: Apr. 2026].

[6] Taoglas, "Cellular Network Infrastructure: Key Components and Their Functions," [Online]. Available: `https://www.taoglas.com/blogs/cellular-network-infrastructure-key-components-and-their-functions/` [Accessed: Apr. 2026].

[7] "LTE Handover — A3 Event Trigger," *Telecommunications for Dummies*, 18 Mar. 2021. [Online]. Available: `https://telecommunications4dummies.com/2021/03/18/example-post-3/` [Accessed: Apr. 2026].

[8] Bright Data, "Mixture of Experts (MoE)," Available: `https://brightdata.com/blog/ai/MixtureofExpertsrts`, accessed Apr. 2026.

[9] Commission for Communications Regulation (ComReg), "SiteViewer: Mobile Mast Locations in Ireland," Available: `https://siteviewer.comreg.ie/#/mobile-masts`, accessed Apr. 2026.

[10] PyTorch Team, "Training Mixture of Experts Models," Available: `https://pytorch.org/blog/training-moes/`, accessed Apr. 2026.